

# PRÁCTICA 3. DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA BASADO EN SERVICIOS WEB CON PYTHON (FASTAPI)

Ibtissam El Amraoui El Houssaini y Candela Libertymeí López Grenville

## 1. Introducción

En esta práctica se ha desarrollado una API REST utilizando Python y el framework FastAPI. El objetivo principal ha sido transformar un sistema basado en objetos distribuidos y métodos remotos en una arquitectura REST accesible mediante HTTP y con intercambio de datos en formato JSON.

El sistema implementado consiste en una API para la gestión de tickets de soporte. A través de distintos endpoints es posible crear tickets, consultarlos, modificar su estado, asignar responsables y añadir comentarios.

Además, se han realizado pruebas utilizando Swagger y Wireshark para comprobar tanto el correcto funcionamiento de la API como el intercambio real de tráfico HTTP.

## 2. Transformación de métodos distribuidos a recursos REST

En la práctica anterior el sistema estaba basado en métodos remotos similares a llamadas RPC. En esta práctica dichos métodos se han transformado en recursos REST utilizando endpoints y métodos HTTP adecuados.

| MÉTODO DISTRIBUIDO                  | ENDPOINT REST                    | MÉTODO HTTP |
|-------------------------------------|----------------------------------|-------------|
| crearTicket()                       | /tickets                         | POST        |
| listarTickets()                     | /tickets                         | GET         |
| consultarTicket(id)                 | /tickets/{ticket_id}             | GET         |
| cambiarEstado(id, estado)           | /tickets/{ticket_id}/estado      | PATCH       |
| asignarResponsable(id, responsable) | /tickets/{ticket_id}/responsable | PATCH       |
| añadirComentario(id, comentario)    | /tickets/{ticket_id}/comentarios | POST        |

Con este diseño se consigue una API más organizada y coherente con la filosofía REST, donde los recursos principales son los tickets y las operaciones se realizan utilizando métodos HTTP estándar.

## 3. Diseño de la API REST

La API se ha implementado utilizando FastAPI debido a su facilidad de uso, integración automática con OpenAPI y generación automática de documentación Swagger.

Los endpoints se han diseñado siguiendo una estructura basada en recursos y evitando acciones tipo RPC. Por ejemplo, en lugar de utilizar rutas como:

**/cambiarEstadoTicket**

se ha utilizado:

**/tickets/{ticket\_id}/estado**

De esta forma la API mantiene una estructura más limpia y escalable.

**API REST de Gestión de Tickets** 1.0.0 GAS 3.1  
/openapi.json  
Servicio REST para gestionar tickets de soporte

| default |                                                    |
|---------|----------------------------------------------------|
| GET     | / Root                                             |
| POST    | /tickets Crear Ticket                              |
| GET     | /tickets Listar Tickets                            |
| GET     | /tickets/{ticket_id} Consultar Ticket              |
| PATCH   | /tickets/{ticket_id}/estado Cambiar Estado         |
| PATCH   | /tickets/{ticket_id}/responsable Asignar Ticket    |
| POST    | /tickets/{ticket_id}/comentarios Añadir Comentario |

#### 4. Uso de HTTP y gestión de errores

Durante el desarrollo se han utilizado distintos métodos HTTP dependiendo de la operación realizada:

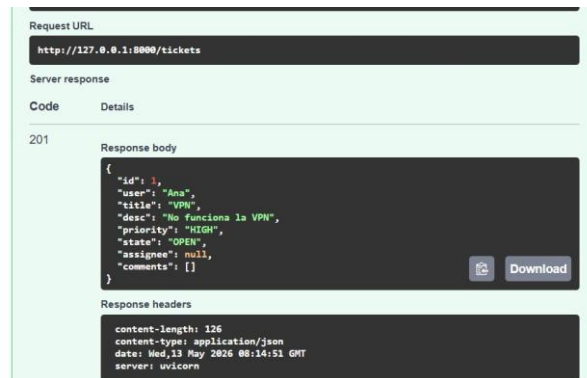
- **GET para consultas.**
- **POST para creación de recursos.**
- **PATCH para modificaciones parciales.**

También se han utilizado códigos de estado HTTP adecuados:

| CÓDIGO        | SIGNIFICADO                  |
|---------------|------------------------------|
| 200 OK        | Operación correcta           |
| 201 Created   | Recurso creado correctamente |
| 404 Not Found | Ticket no encontrado         |

##### ➤ CREACIÓN DE TICKETS

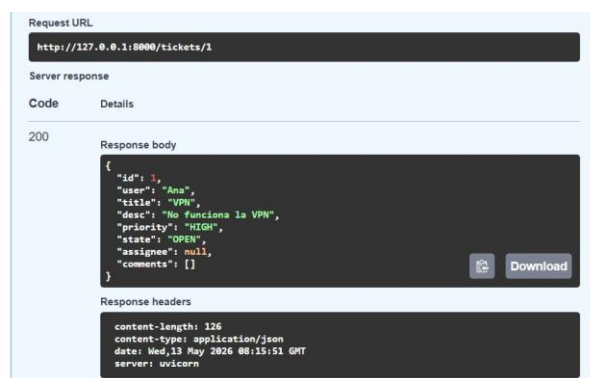
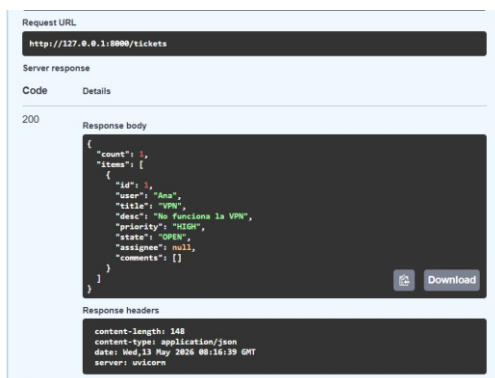
La creación de tickets se realiza mediante el método HTTP POST sobre el recurso **/tickets**.



**Figura 2.** Creación correcta de un ticket utilizando POST y devolución del código HTTP 201 Created.

##### ➤ CONSULTA DE TICKETS

La API permite consultar tanto la lista completa de tickets como un ticket individual mediante su identificador.



**Figura 3 y 4.** Consulta de todos los tickets almacenados mediante una petición GET y consulta individual de un ticket utilizando su identificador.

##### ➤ MODIFICACIÓN DEL ESTADO

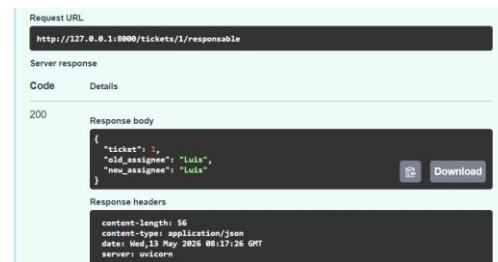
Para modificar parcialmente recursos ya existentes se ha utilizado el método PATCH.



**Figura 5.** Cambio de estado de un ticket utilizando el método PATCH.

### ➤ ASIGNACIÓN DE RESPONSABLE

La API permite asignar responsables a los tickets utilizando un subrecurso específico.



**Figura 6.** Asignación de un responsable a un ticket.

### ➤ AÑADIR COMENTARIOS

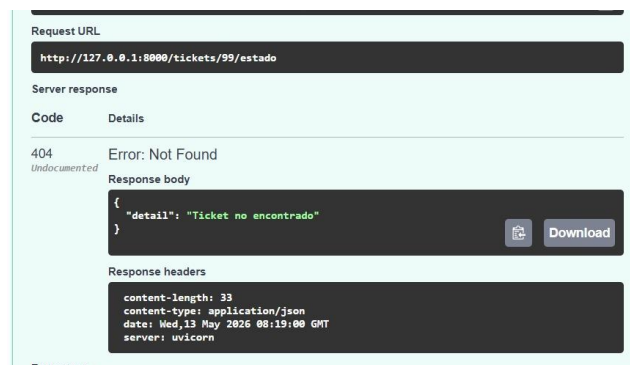
Los comentarios asociados a un ticket se añaden mediante una petición POST al subrecurso /comentarios.



**Figura 7.** Inserción de comentarios asociados a un ticket.

### ➤ GESTIÓN DE ERRORES

La API implementa gestión de errores mediante códigos HTTP adecuados.



**Figura 8.** Gestión de errores cuando se intenta acceder a un ticket inexistente.

## 5. Captura y análisis del tráfico HTTP

Para comprobar el funcionamiento real de la comunicación cliente-servidor se utilizó Wireshark capturando tráfico sobre el puerto 8000.

En las capturas se pueden observar peticiones HTTP GET, POST y PATCH junto con sus correspondientes respuestas JSON generadas por la API.

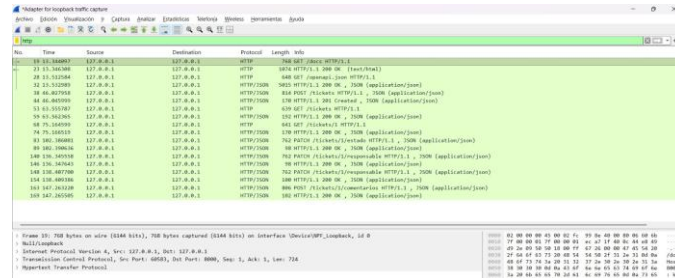


Figura 9. Captura de tráfico HTTP generado durante el uso de la API REST.

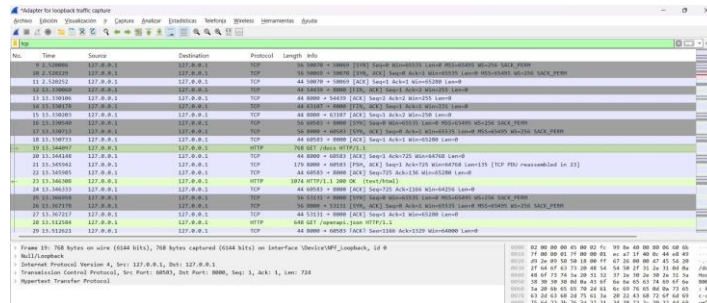


Figura 10. Captura de paquetes TCP utilizados durante la comunicación cliente-servidor.

## 6. Ventajas e inconvenientes de REST

### ➤ VENTAJAS

- Arquitectura más simple y estándar.
- Uso directo de HTTP.
- Fácil integración con clientes y aplicaciones web.
- Respuestas en JSON fáciles de interpretar.
- Documentación automática mediante Swagger.

### ➤ INCONVENIENTES

- El sistema es stateless, por lo que el servidor no mantiene estado entre peticiones.
- Algunas operaciones complejas requieren varios endpoints.
- Mayor dependencia de la estructura de URLs y métodos HTTP.

## 7. Conclusiones

La práctica ha permitido comprender la transición desde un sistema basado en objetos distribuidos hacia una arquitectura REST utilizando FastAPI.

Se ha aprendido a diseñar endpoints correctamente, utilizar métodos HTTP adecuados y documentar automáticamente la API mediante Swagger/OpenAPI.

Además, gracias a Wireshark se ha podido comprobar el intercambio real de tráfico HTTP entre cliente y servidor, verificando el correcto funcionamiento de la aplicación desarrollada.